

Join the lecture online on your dashboard.



From RNN Limitations to Seq2Seq

Encoder–Decoder Architecture

Recap – Sentiment classification

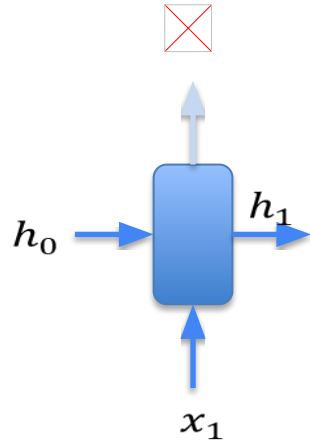
Text: “Dhurandhar
is too good!”

RNN has to find
the sentiment from
this text.

(Many-to-one)

Recap – Sentiment classification

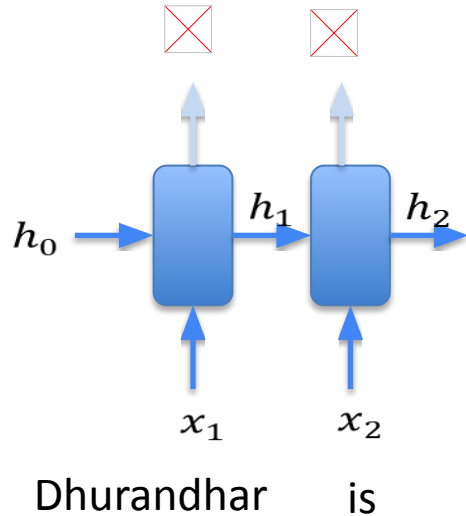
- Text: “Dhurandhar is too good!”



Dhurandhar

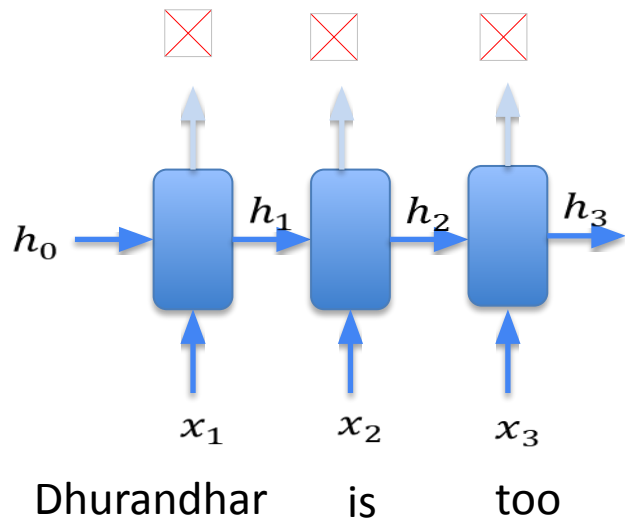
Recap – Sentiment classification

- Text: “Dhurandhar is too good!”



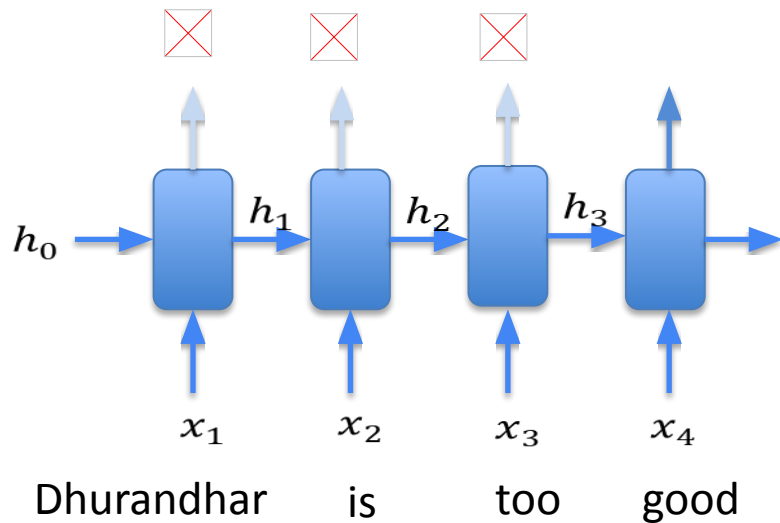
Recap – Sentiment classification

- Text: “Dhurandhar is too good!”



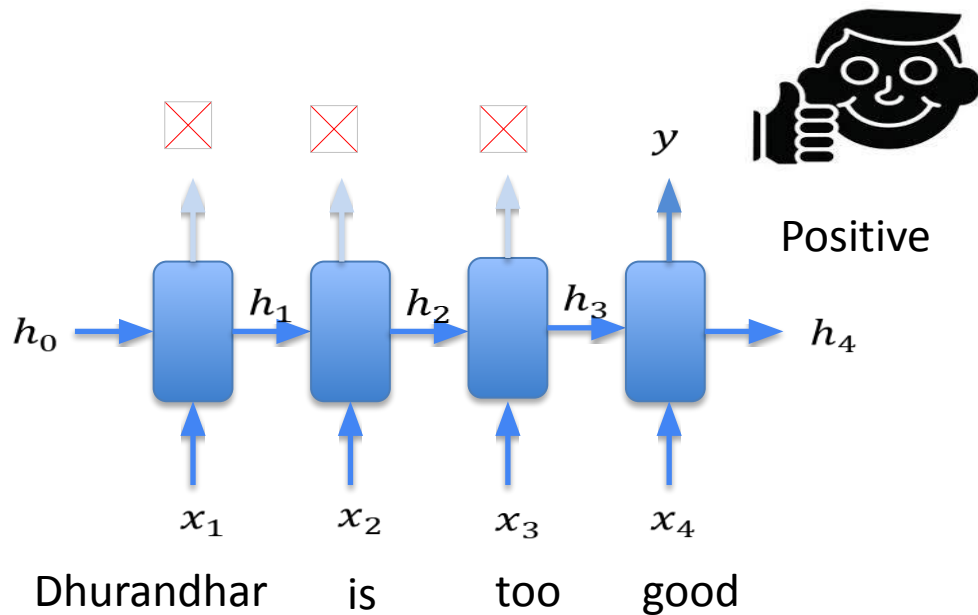
Recap – Sentiment classification

- Text: “Dhurandhar is too good!”



Recap – Sentiment classification

- Text: “Dhurandhar is too good!”



Recap – Parts of speech tagging

Text: “The cat is
sleeping.”

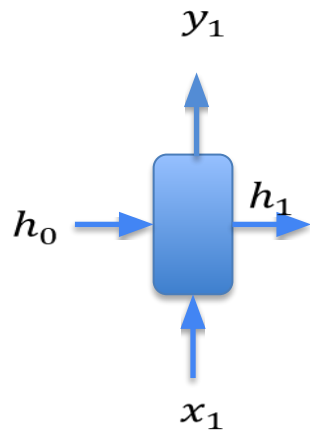
RNN has to tag a
grammatical label to
each word.

(Many-to-many)

Recap – Parts of speech tagging

- Text: “The cat is sleeping.”

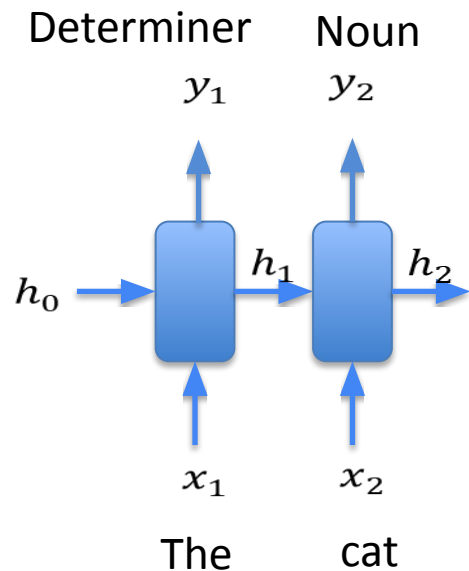
Determiner



The

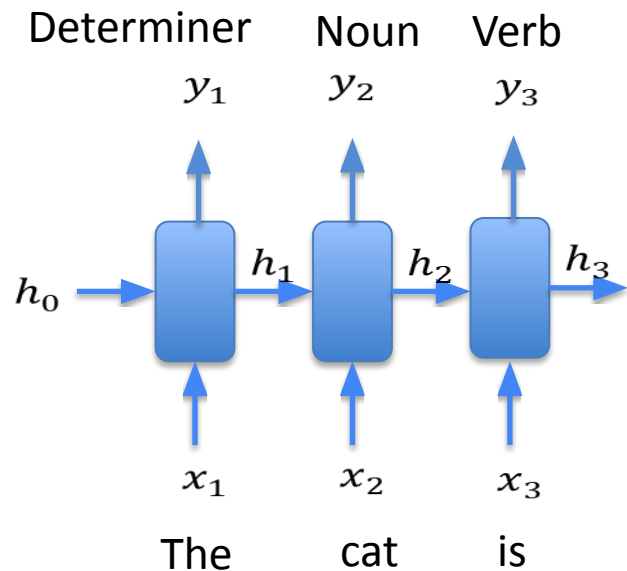
Recap – Parts of speech tagging

- Text: “The cat is sleeping.”



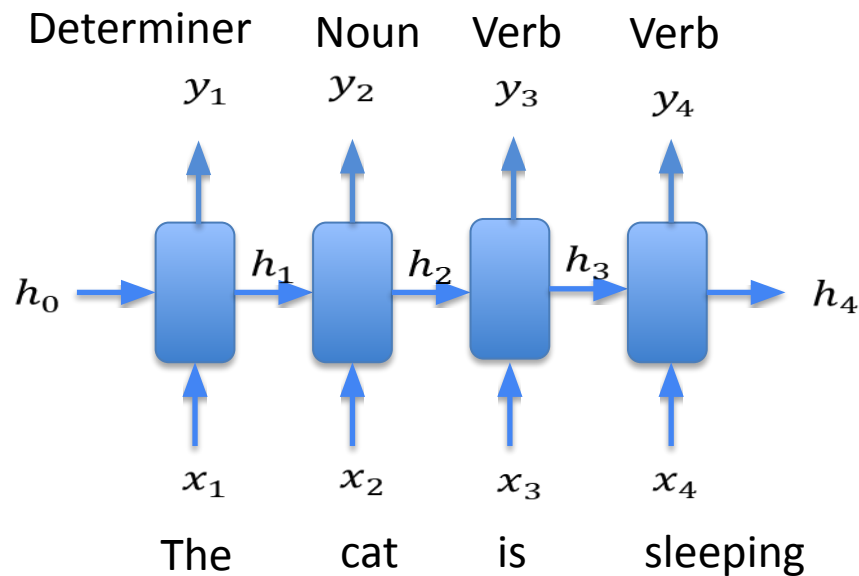
Recap – Parts of speech tagging

- Text: “The cat is sleeping.”



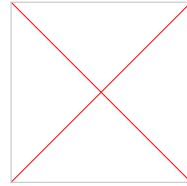
Recap – Parts of speech tagging

- Text: “The cat is sleeping.”



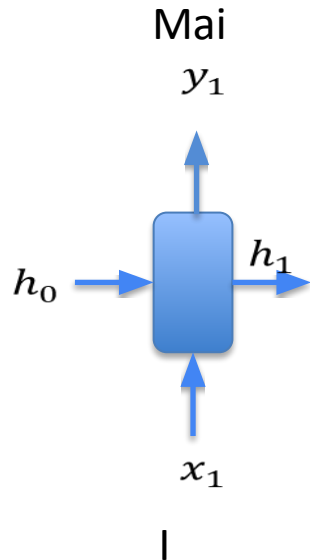
Machine Translation

- Translate: 'I am going.' → 'Mai ja raha hu.'
- RNN has to translate from English to Hinglish.



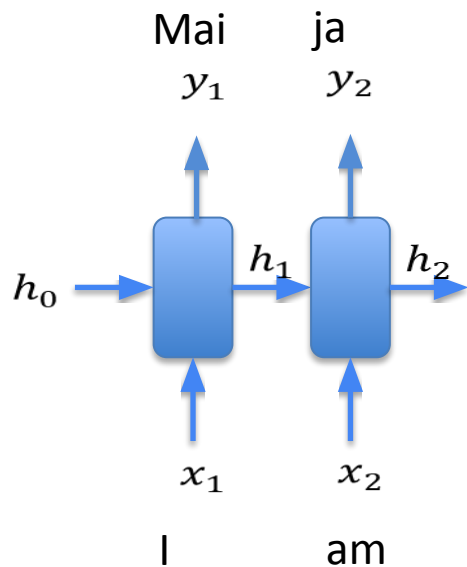
Machine Translation

- Translate: 'I am going.' $\rightarrow$ 'Mai ja raha hu.'



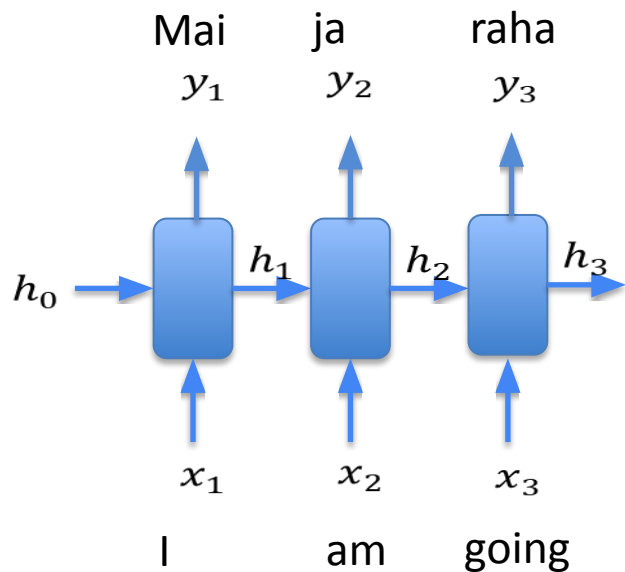
Machine Translation

- Translate: 'I am going.' $\rightarrow$ 'Mai ja raha hu.'



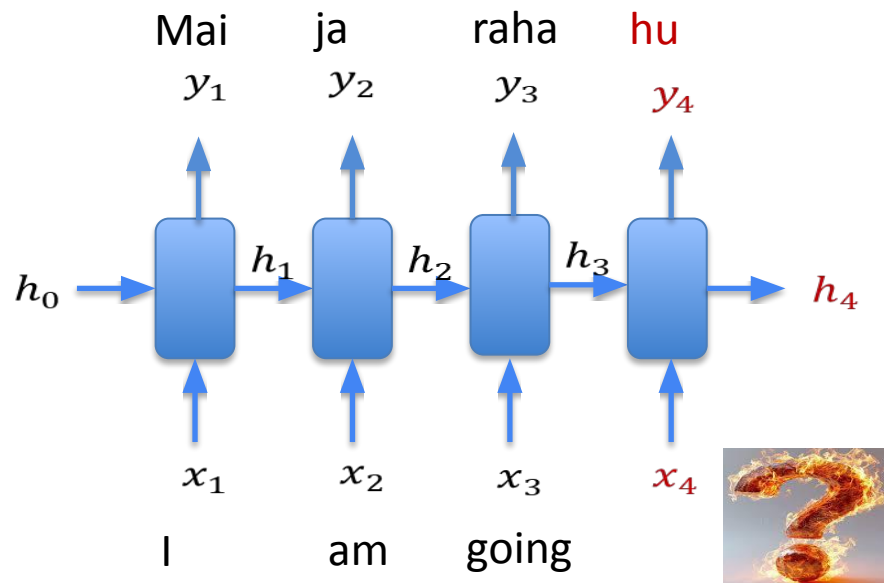
Machine Translation

- Translate: 'I am going.' $\rightarrow$ 'Mai ja raha hu.'



Machine Translation

- Translate: 'I am going.' $\rightarrow$ 'Mai ja raha hu.'



Sequence-to-Sequence

- Why do you think it failed??

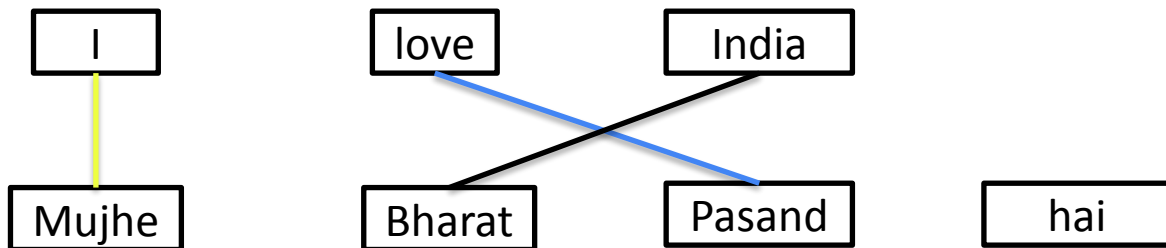
Sequence-to-Sequence

- Why do you think it failed??
- Input length $\neq$ Output length

Sequence-to-Sequence

● Why do you think it failed??

- Input length $\neq$ Output length
- No fixed alignment.



RNN Limitation

- Translate: “I love India.” → “Mujhe Bharat pasand hai.”

- Inputs:

I

love

India

- Outputs:

Mujhe

Bharat

Pasand

hai

- When is the best time to output “Mujhe”.

RNN Limitation

- Translate: “I love India.” → “Mujhe Bharat pasand hai.”

• Inputs: I love India

• Outputs: Mujhe Bharat Pasand hai

- After seeing entire sentence?
- Or after seeing “I”?



RNN Limitation

- Translate: “I love India.” → “Mujhe Bharat pasand hai.”

• Inputs:

I

love

India

• Outputs:

Mujhe

Bharat

Pasand

hai

• After seeing entire sentence?



• Or after seeing “I”?

RNN Limitation

One input $\rightarrow$ one
output

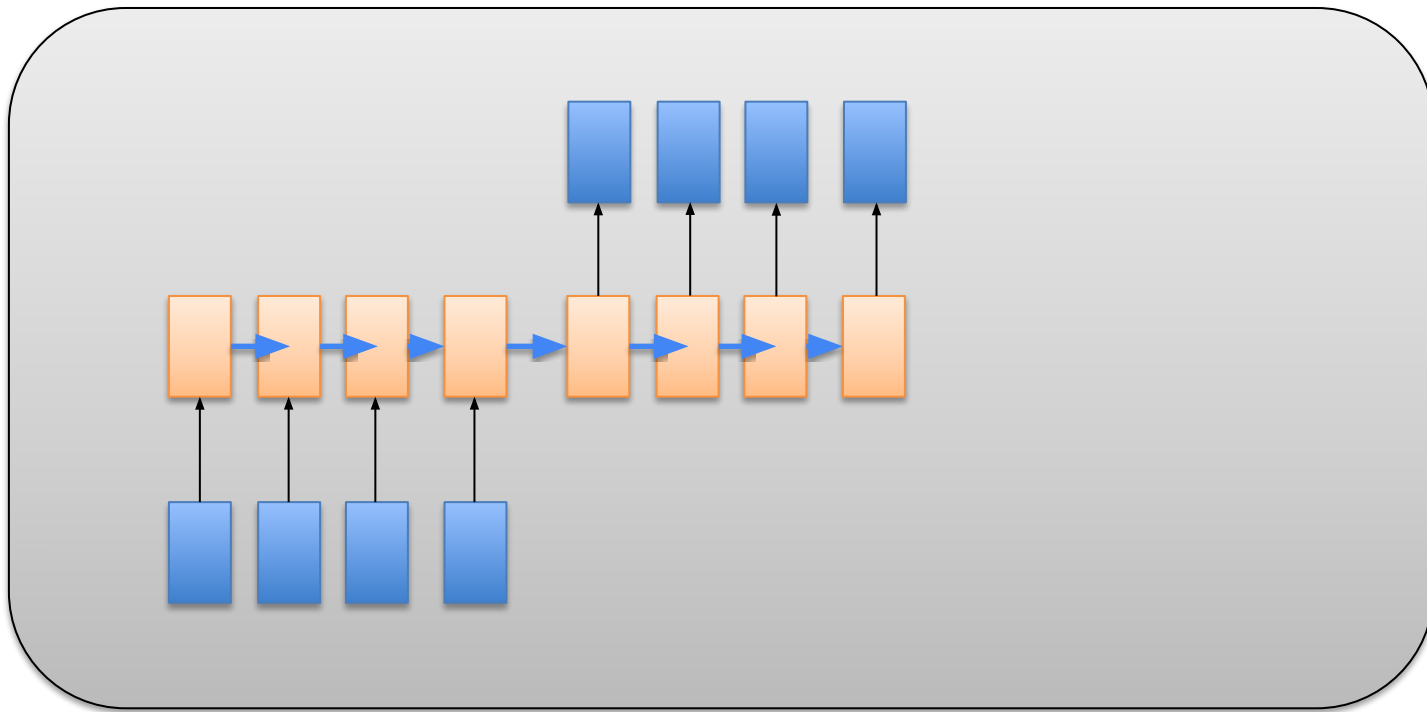
Cannot handle
variable-length
mapping

No global
understanding

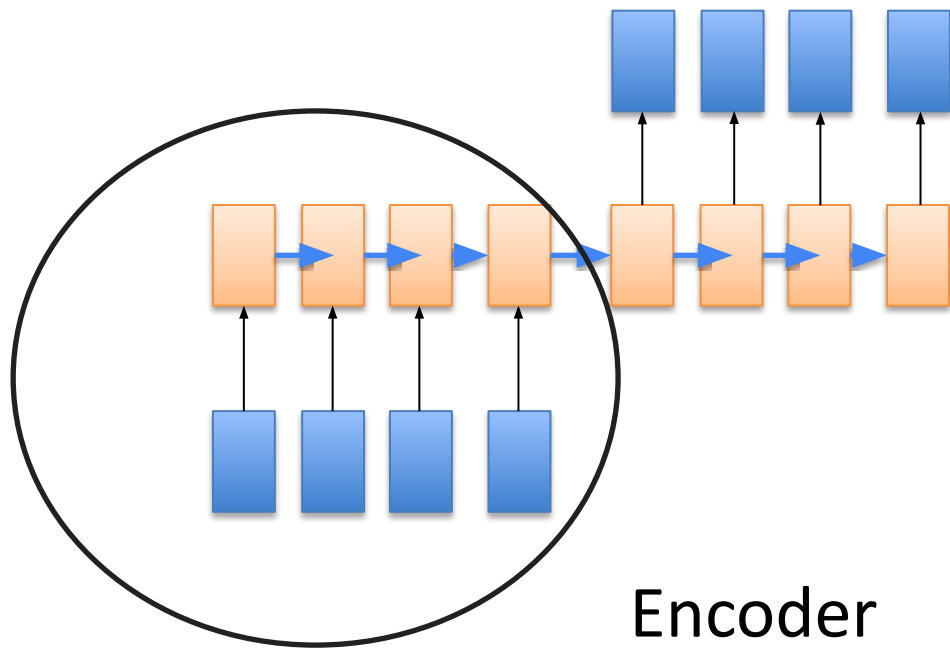
Use of Sequence-to-Sequence modelling

- Machine Translation
- Chatbots
- Speech Recognition
- Text Summarization

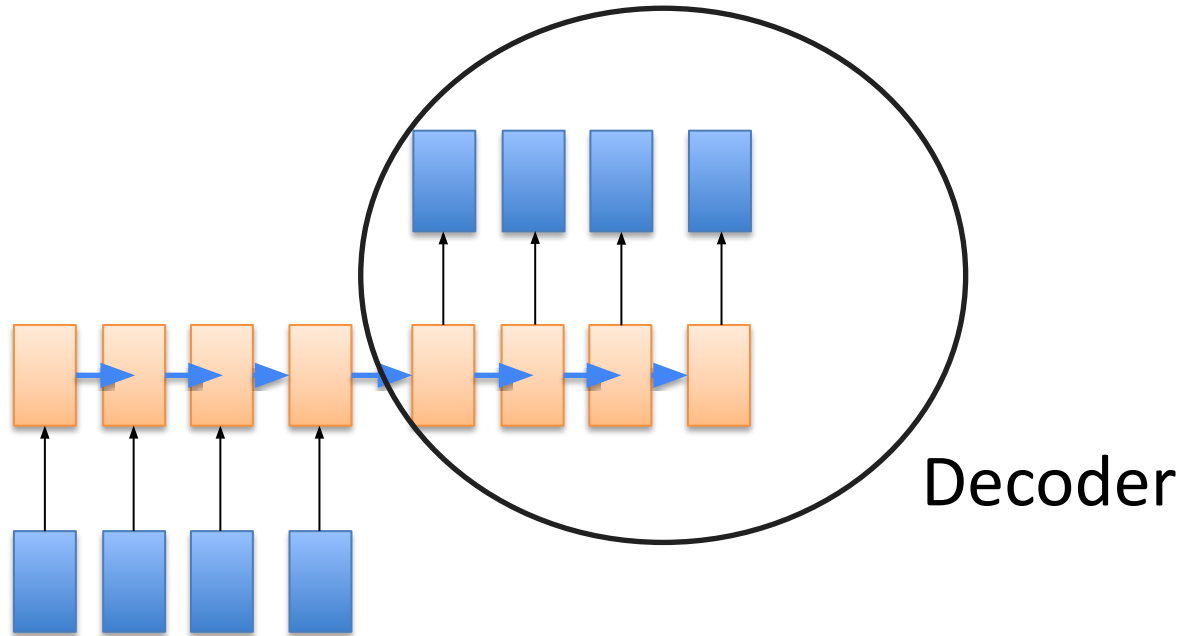
Intuitive solution – *Delayed* sequence-to-sequence



Intuitive solution – *Delayed* sequence-to-sequence

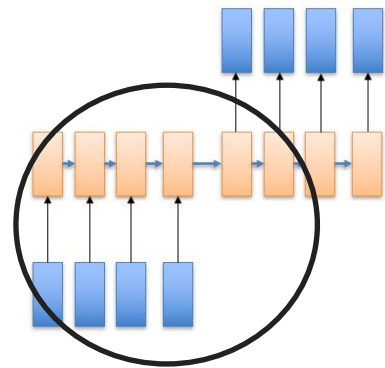


Intuitive solution – *Delayed* sequence-to-sequence



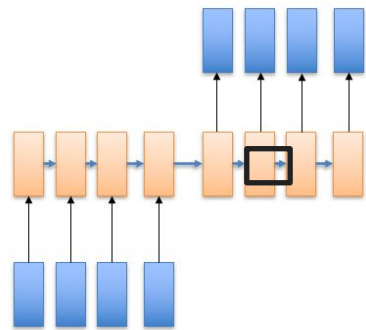
Intuitive solution – *Delayed* sequence-to-sequence

- Encoder reads entire sequence.
- Encoder generates a vector representation of the input.



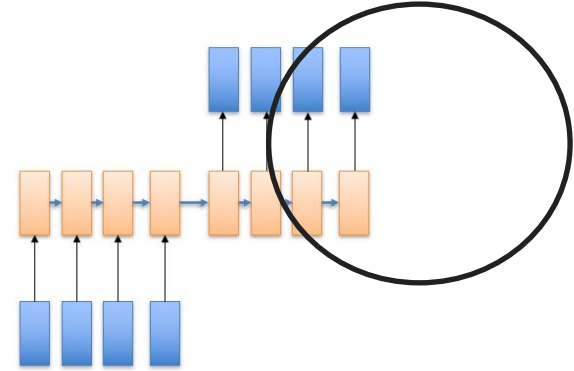
Intuitive solution – *Delayed* sequence-to-sequence

- Encoder reads entire sequence.
- Encoder generates a vector representation of the input.
- This vector contains the context or semantic meaning of the input.

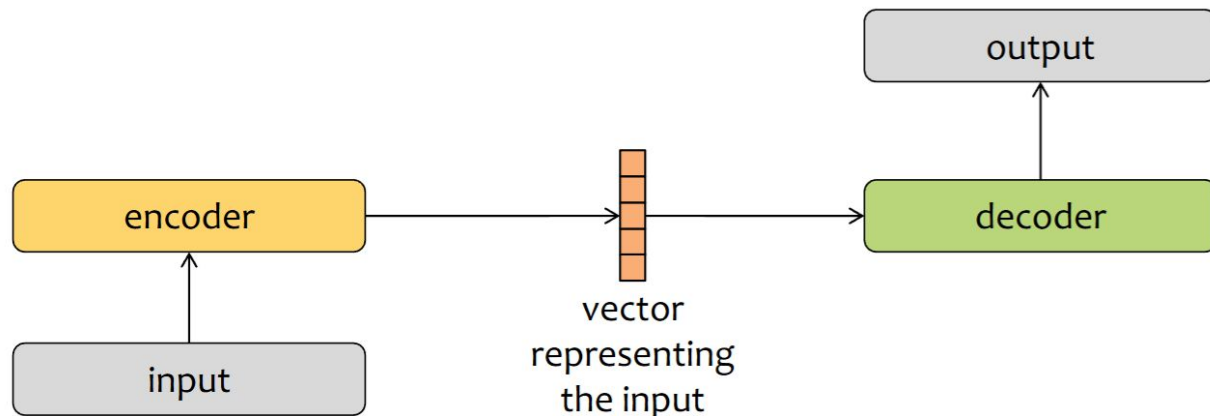


Intuitive solution – *Delayed* sequence-to-sequence

- Encoder reads entire sequence.
- Encoder generates a vector representation of the input.
- This vector contains the context or semantic meaning of the input.
- Decoder takes this representation as an input.
- Decoder produces one output at a time.



Encoder-Decoder



Sequence to sequence learning with neural networks

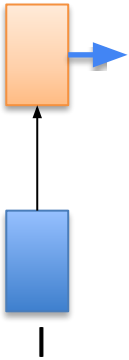
[I Sutskever](#), [O Vinyals](#), [QV Le](#) - *Advances in neural ...*, 2014 - proceedings.neurips.cc

Abstract Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then ...

☆ Save 📄 Cite Cited by 31452 Related articles All 41 versions 🔗

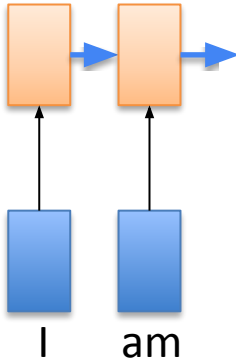
Training Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



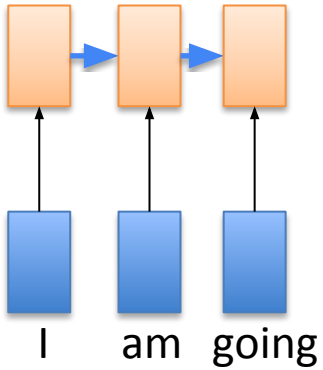
Training Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



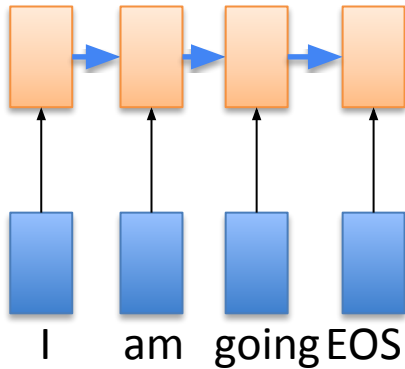
Training Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



Training Phase

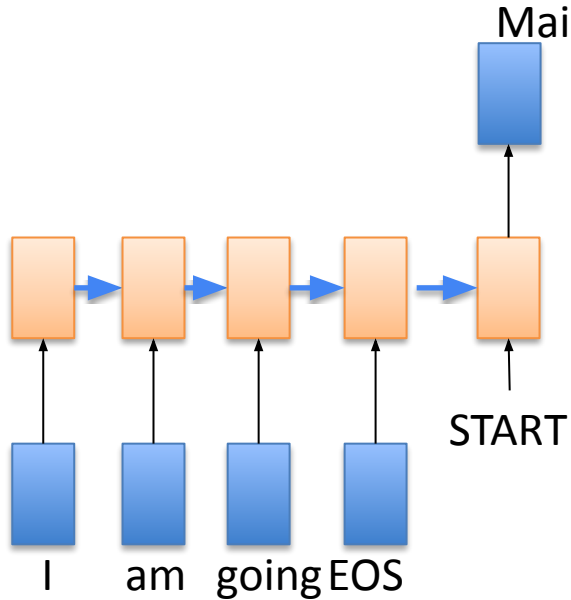
- Translate: 'I am going.' → 'Mai ja raha hu.'



EOS – It is a special token, indicating the end of input sentence.

Training Phase

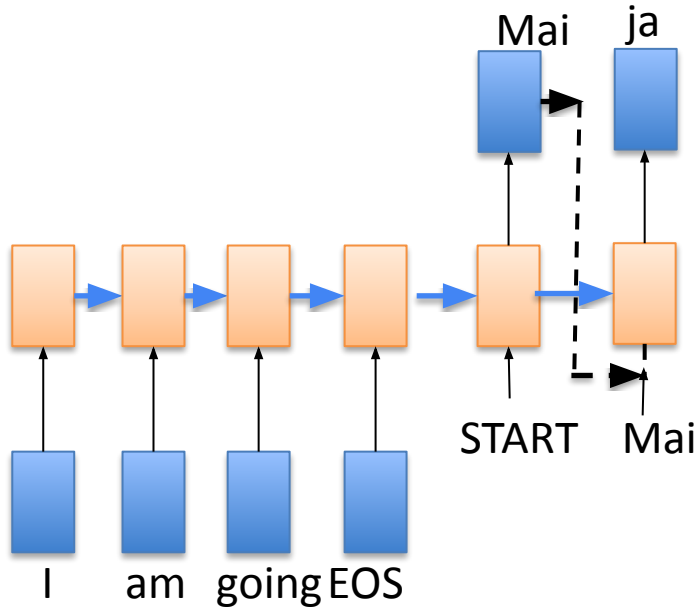
- Translate: 'I am going.' → 'Mai ja raha hu.'



START is also a special token to start decoding process.

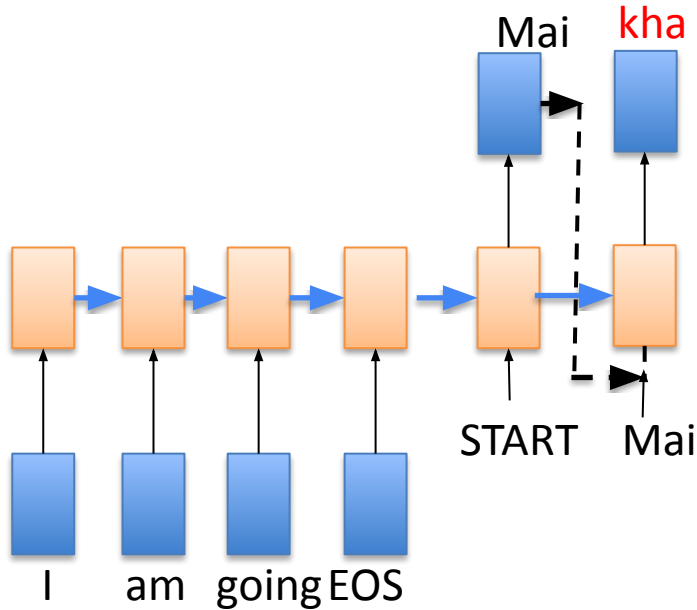
Training Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



Training Phase

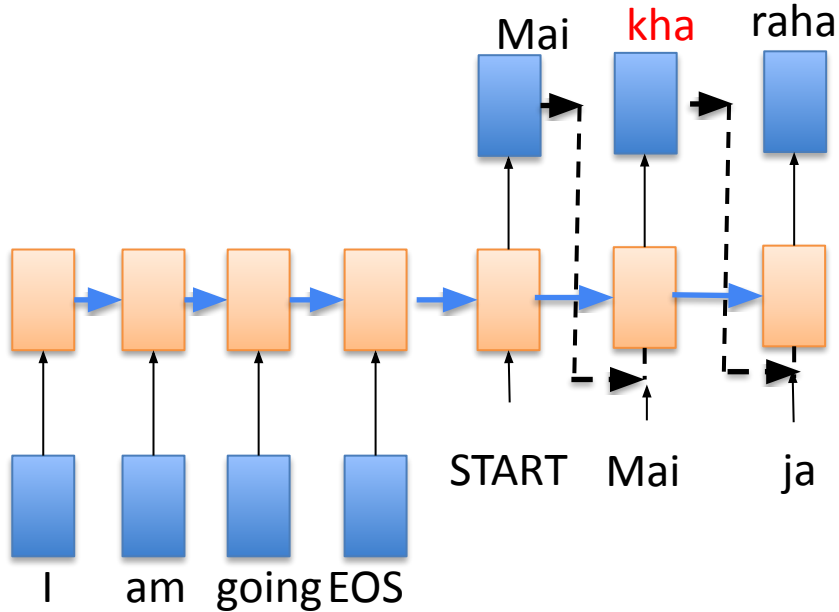
- Translate: 'I am going.' → 'Mai ja raha hu.'



- Let the decoder produces **wrong word**, we input only **correct word** in the next time step of the **decoder**.
- Because we already have the **correct** output.
- Loss will be calculated accordingly.

Training Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



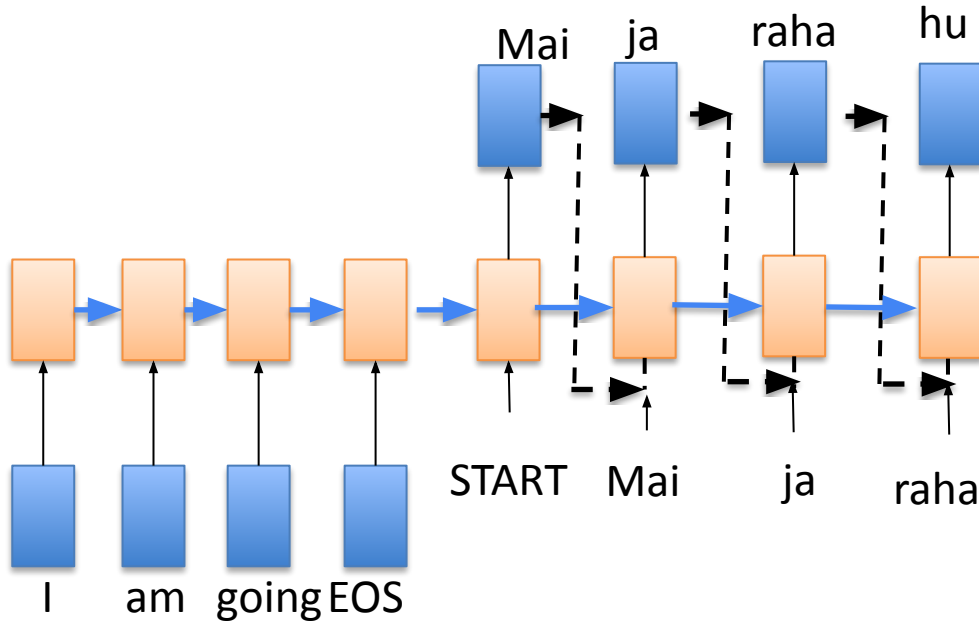
This is called teacher forcing.

Teacher Forcing

- Uses actual output during training.
- Leads to faster convergence.

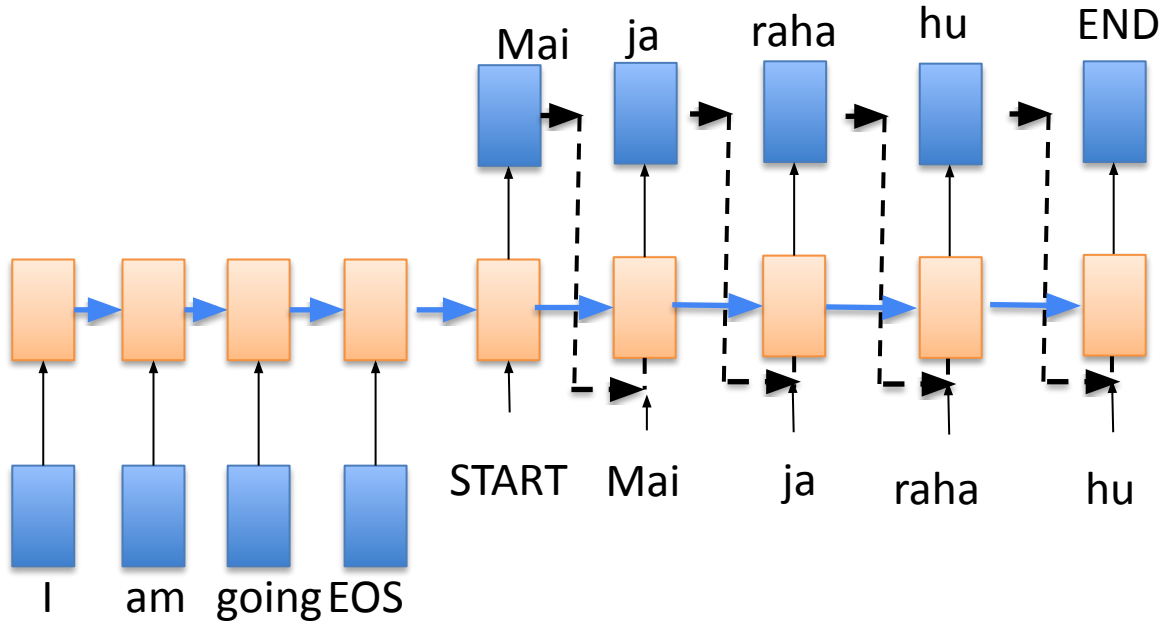
Training Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



Training Phase

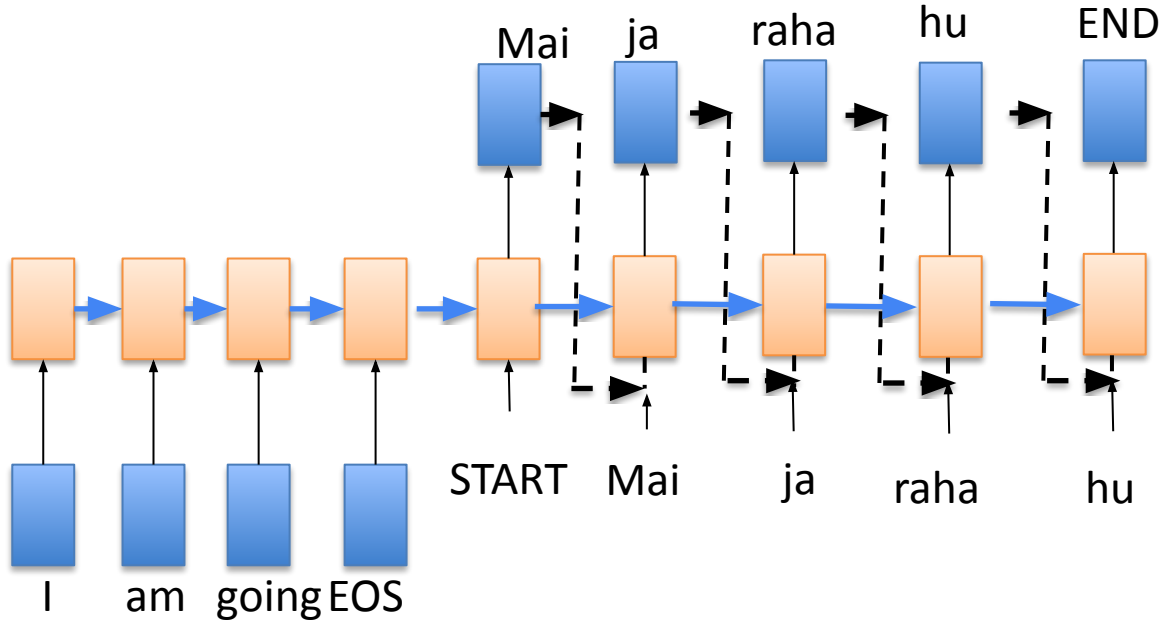
- Translate: 'I am going.' → 'Mai ja raha hu.'



END – Special token to indicate end of output.

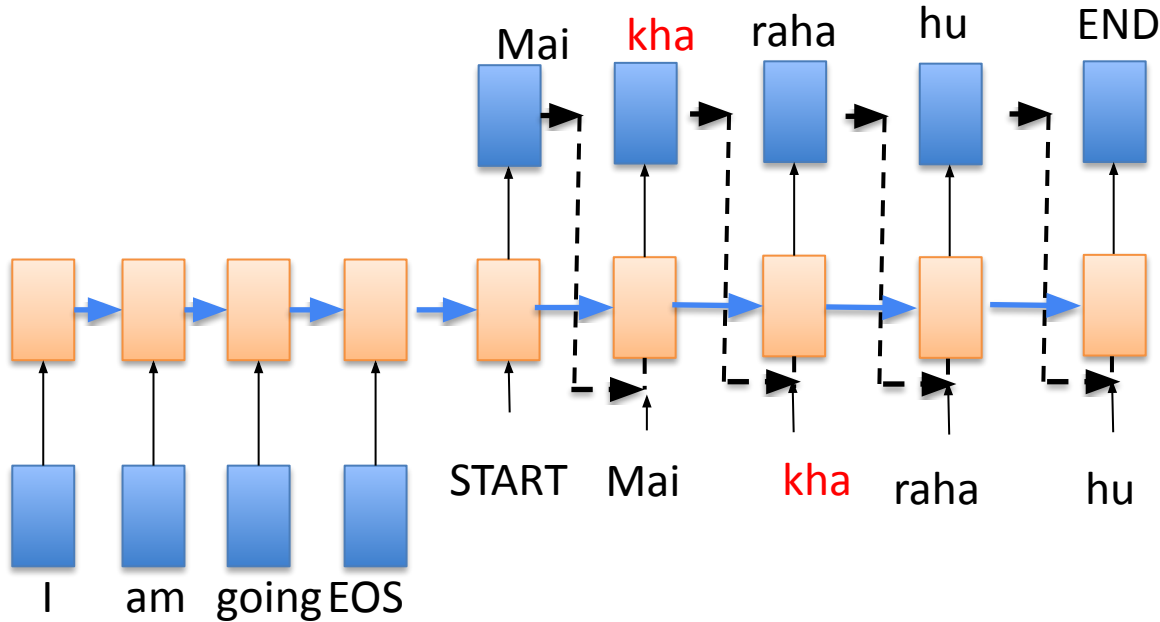
Prediction Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



Prediction Phase

- Translate: 'I am going.' → 'Mai ja raha hu.'



If there is a mistake in prediction, that will be carried to next time step.

Encoder-Decoder

- Encoder and Decoder are separate RNNs.
- Encoder-Decoder by Sutskever uses LSTMs.
- GRUs can also be used.

LSTM / GRU – Improvement?

- Handles long dependencies.
- Better memory.
- Still compresses sequence into ONE vector.

The Bottleneck Problem

- Entire sequence $\rightarrow$ fixed-size vector.
- Information loss increases with length.
- Performance drops for long sequences.

Thought Experiment

- Summarize a paragraph in ONE word
- Will meaning be lost? → YES
- Same issue in RNN/LSTM/GRU

Key Idea of Seq2Seq

- Separate understanding and generation.
- Encoder + Decoder architecture.

Thinking.....

- What does context vector contain?
- Meaning? Grammar? Relationships?
- Everything compressed into ONE vector.

Hidden Problem

- Single vector must store entire sentence.
- Difficult for long/complex sequences.

Think Again

- What if decoder accesses ALL encoder states?

Coming up....

- Attention Mechanism

Attention is all you need

[A Vaswani](#), [N Shazeer](#), [N Parmar](#)... - Advances in neural ..., 2017 - [proceedings.neurips.cc](#)

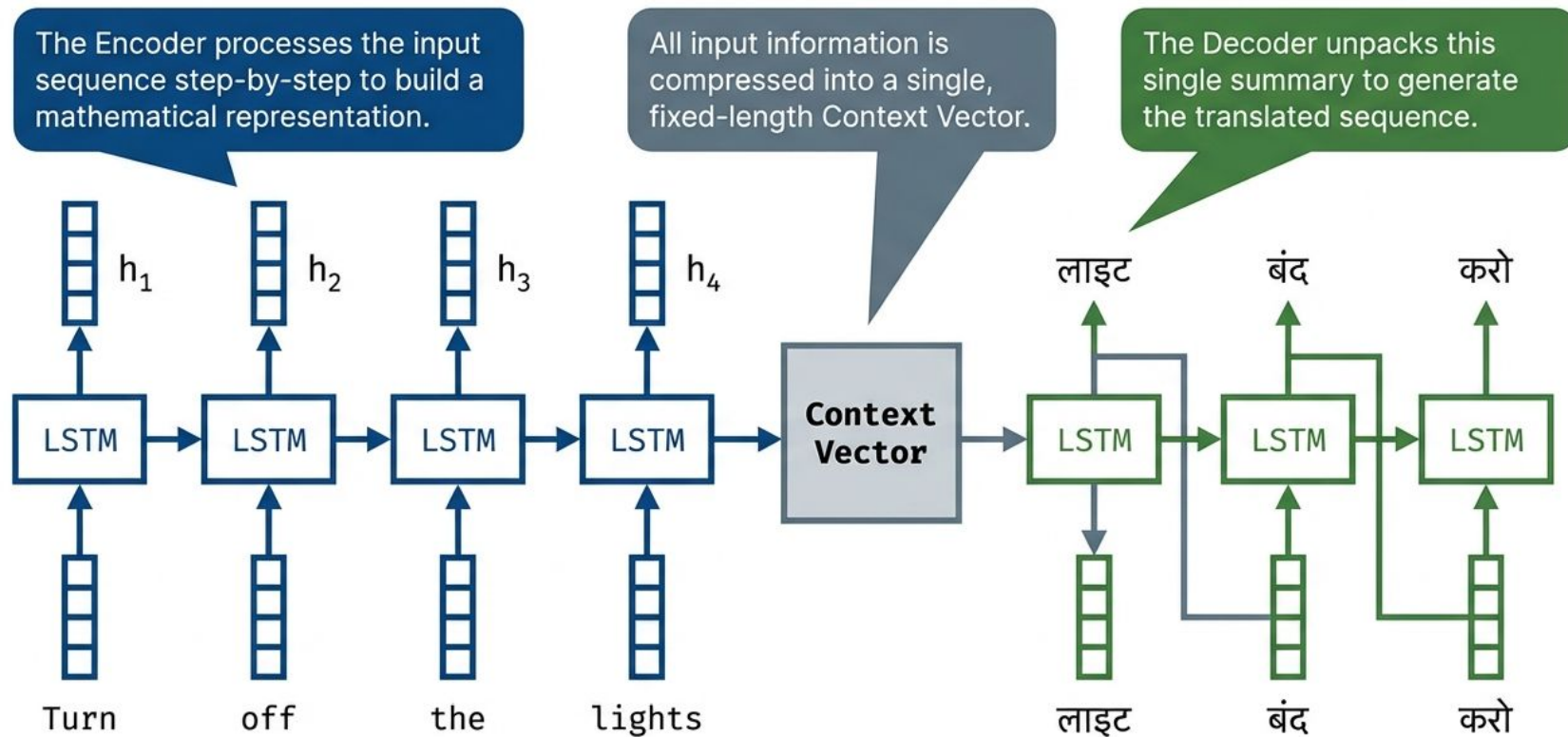
... to attend to **all** positions in the decoder up to and including that position. **We need** to prevent

... **We** implement this inside of scaled dot-product **attention** by masking out (setting to $-\infty$) ...

☆ Save  Cite Cited by 238029 Related articles All 71 versions 

Background of Attention mechanism

The Baseline: Sequence-to-Sequence NMT



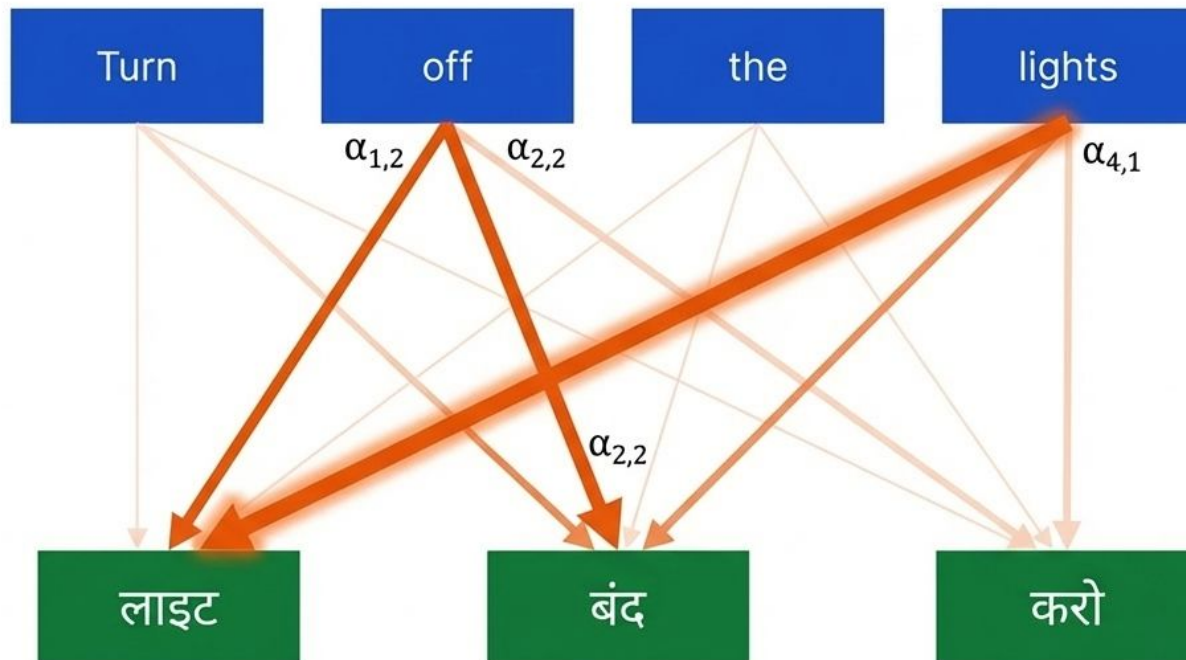
The Paradigm Shift: Dynamic Alignment

Key Insight:

To predict a specific word, the Decoder doesn't need the whole sentence—it needs a specific subset of words.

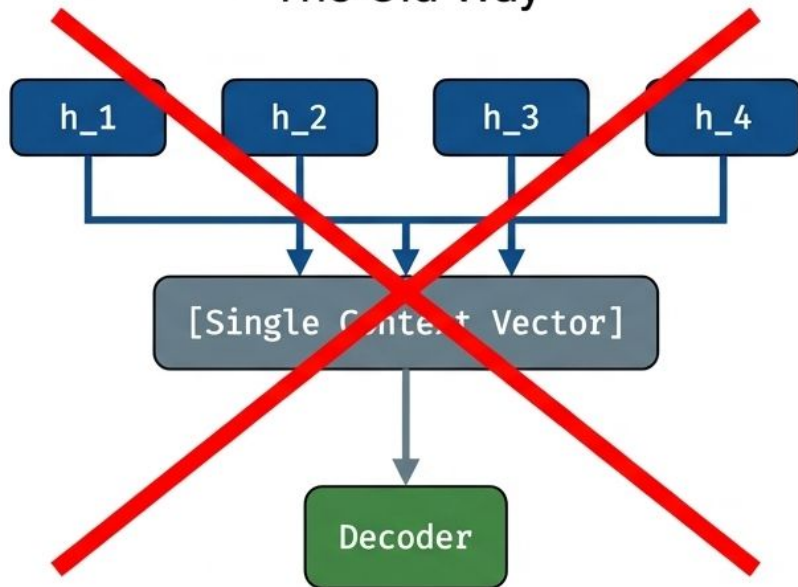
Mechanism:

Instead of one static summary, we dynamically calculate Alignment Scores to determine the weight of each input word for the current output word.



From Static Summary to Dynamic Access

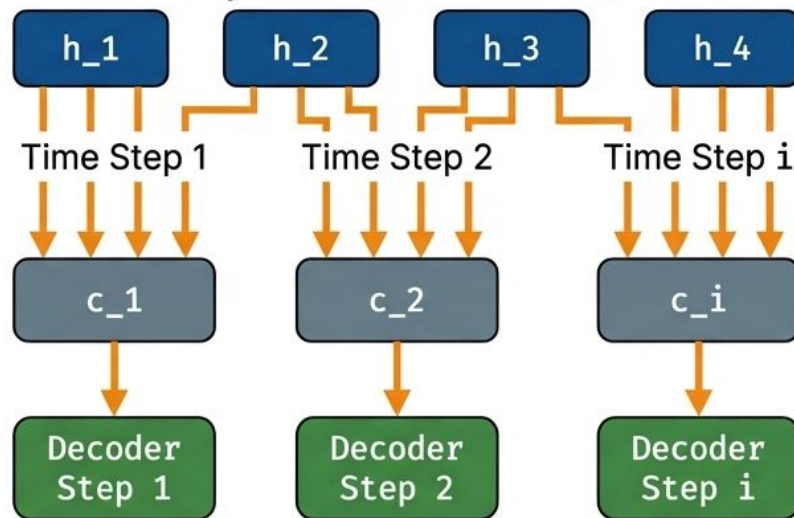
The Old Way



Rule: Every intermediate hidden state of the Encoder ($h_1 \dots h_4$) remains fully available at every time step of the Decoder.

The Attention Way

Memory: All Encoder States Available



Result: The Decoder generates a unique Context Vector (c_i) for every output word it predicts.

Two Types of Attention

Bahdanau Attention

Bahdanau attention, also known as Additive attention.

It proposed a more complex way than just using a mathematical approach to compute the attention weights by conducting a linear combination of encoder and decoder states.

Bahdanau technique employs and trains a feed-forward neural network. Moreover, the current state of the decoder, the previous state of the attention mechanism, and the current input are driven through the neural network.

Next, the attention weights are utilized to produce the weighted sum of the features of the input. This weighted sum is provided as additional information to the decoder, allowing it to focus on important input factors when producing a single output.

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

Mathematical approach

Specifically, the attention weights are calculated using the following equation.

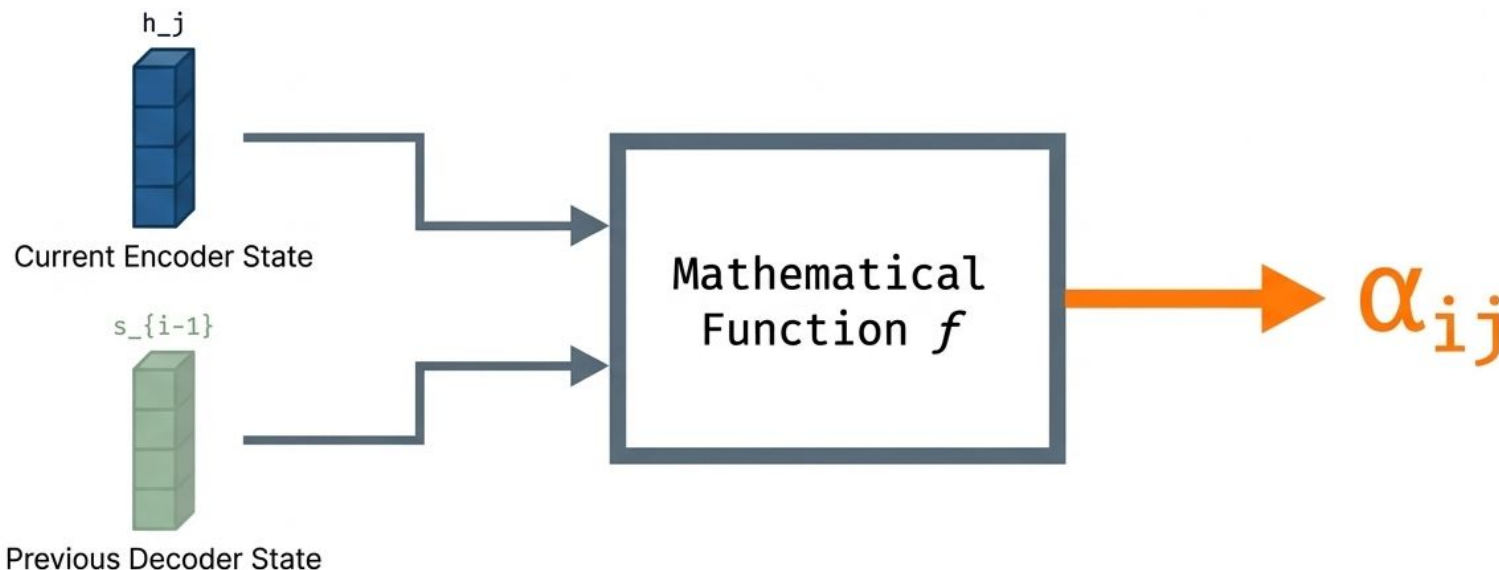
$$attention_weights = softmax(V_a * tanh(W_a * encoder_states + U_a * decoder_state + b_a))$$

Where *encoder_states* is a matrix containing the hidden states of the encoder, *decoder_state* is the current hidden state of the decoder, V_a , W_a , U_a , and b_a are learned parameters, and *tanh* is the hyperbolic tangent function.

Bahdanau attention can learn more complex relations between the data than other types of attention mechanisms because it employs a neural network to compute the attention weights rather than a simple mathematical algorithm.

It is also a powerful technique as it allows the attention weights to be learned directly from the data.

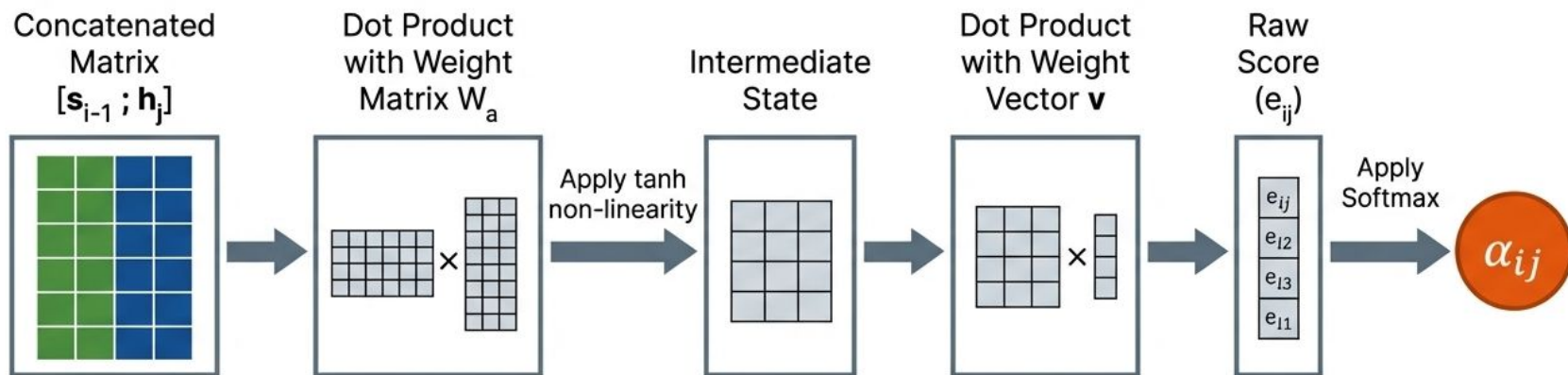
Bahdanau “Additive” Attention: The Core Hypothesis



The Premise	The Bahdanau Solution
To figure out what to focus on next, the system needs to know what it is currently looking at (h_j) AND what it just finished translating (s_{i-1}).	Instead of guessing a complex mathematical function for f , Bahdanau approximates it using a Feed-Forward Neural Network (FFNN).

Bahdanau: Tracking the Tensors

$$\alpha_{ij} = \text{Softmax}(\mathbf{v}^T \tanh(W_a[\mathbf{s}_{i-1}; \mathbf{h}_j] + \mathbf{b}))$$



Summary: A single hidden layer seamlessly maps the 8-dimensional concatenated state down to raw scalar importance scores (e_{ij}), ready for normalization.

Luong Attention

Luong attention, also known as scaled dot-product attention.

This technique's attention weights are generated by employing a set of standard scoring algorithms that use as input the **decoder's actual hidden state and the **encoder's** hidden states as well.** These algorithms yield a score that indicates each encoder state's "impact" in creating the current decoder state. Therefore, the produced scores are then used to compute the attention weights.

General attention computes the dot product between the attention matrix and the current hidden state, which results in a single scalar: $attention_weights = softmax(V_a * (encoder_states * decoder_state))$

Concatenation attention concatenates the current hidden state with each encoder hidden state and applies a feed-forward neural network to compute the attention scores:

$$attention_weights = softmax(V_a * W_a * [encoder_states; decoder_state])$$

Dot attention is the simplest form, which computes the dot product between the current hidden state and each encoder hidden state, which results in attention scores:

Came out in
2015

[cs.CL] 20 Sep 2015

Effective Approaches to Attention-based Neural Machine Translation

Minh-Thang Luong Hieu Pham Christopher D. Manning
Computer Science Department, Stanford University, Stanford, CA 94305
{lmthang, hyhieu, manning}@stanford.edu

Abstract

An attentional mechanism has lately been used to improve neural machine translation (NMT) by selectively focusing on parts of the source sentence during translation. However, there has been little work exploring useful architectures for attention-based NMT. This paper examines two simple and effective classes of attentional mechanism: a *global* approach which always attends to all source words and a *local* one that only looks at a subset of source words at a time. We demonstrate the effectiveness of both approaches on the WMT translation tasks between English and German in both directions. With local

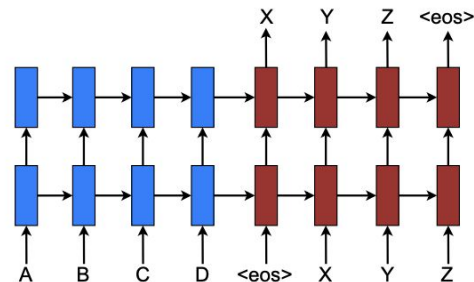
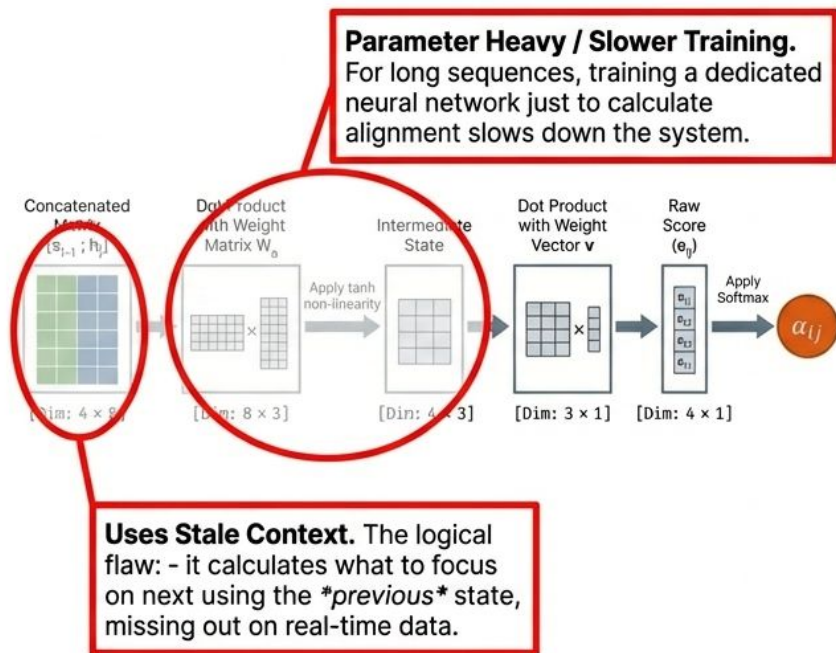


Figure 1: **Neural machine translation** – a stacking recurrent architecture for translating a source sequence A B C D into a target sequence X Y Z. Here, `<eos>` marks the end of a sentence.

emitting one target word at a time, as illustrated in

The Refactor: Why Luong Attention?

The Bahdanau Bottleneck



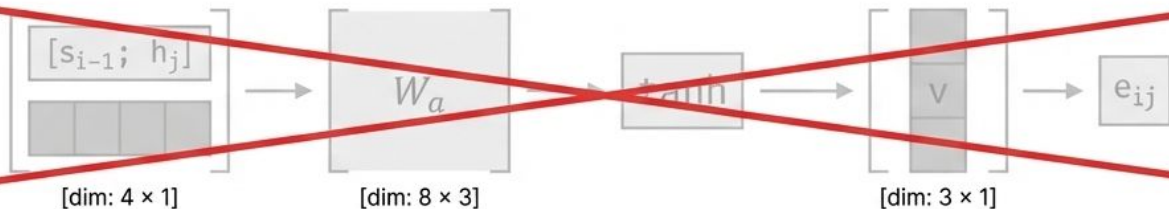
The Luong Objective

Goal 1: Eliminate the Neural Network.
Can we compute similarity faster using raw vector geometry?

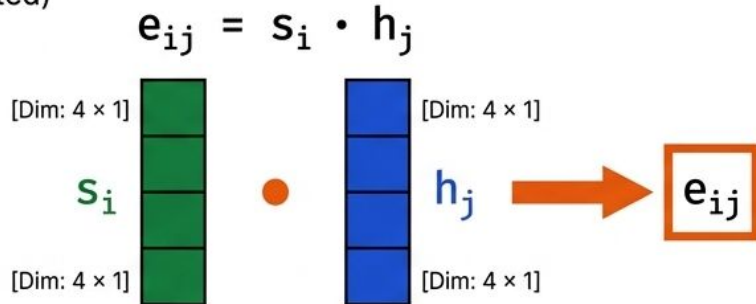
Goal 2: Immediate Context.
Can we dynamically adjust attention using the *current* decoder state?

Luong “Multiplicative” Attention: The Dot Product

Bahdanau FFNN Pipeline (Eliminated)



Luong “Multiplicative” Attention (Spotlighted)



1

The Insight

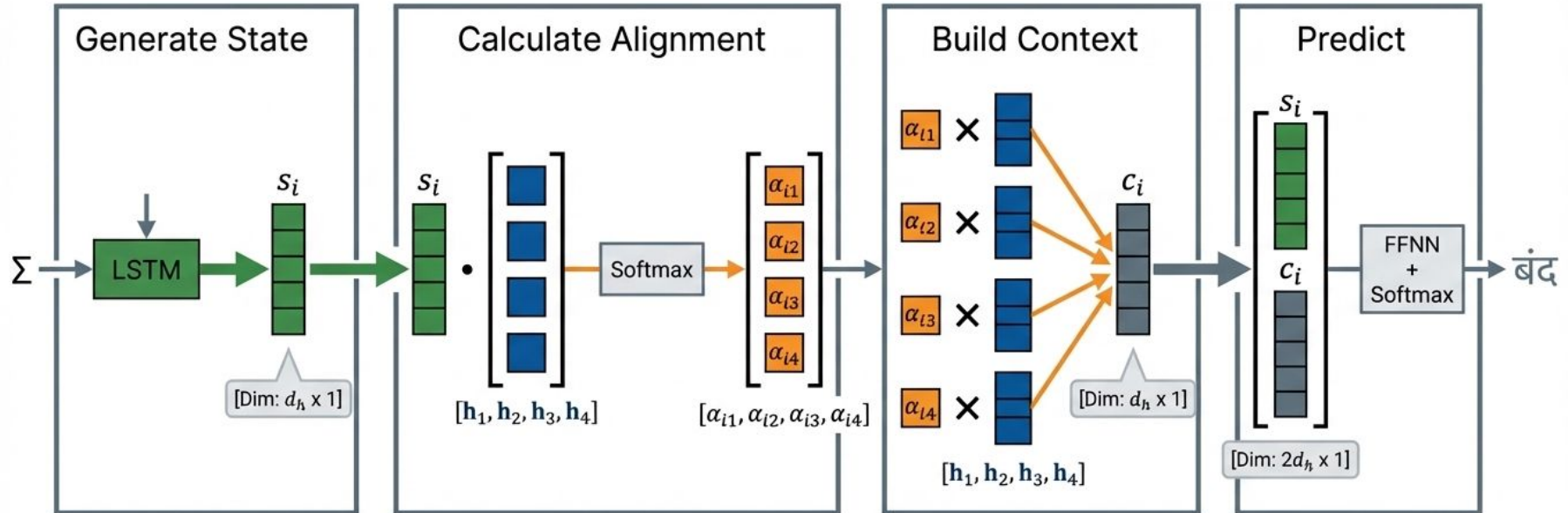
If vectors are similar, their dot product is large. If dissimilar, it's small. Similarity is natively encoded in geometry.

2

The Engineering Win

By ensuring s_i and h_j share the same dimensionality, Luong completely eliminates the W_a parameter matrix. The entire alignment neural network is erased.

The Complete Luong Pipeline



A streamlined, mathematically efficient sequence that maximizes recent context while minimizing trainable parameters.

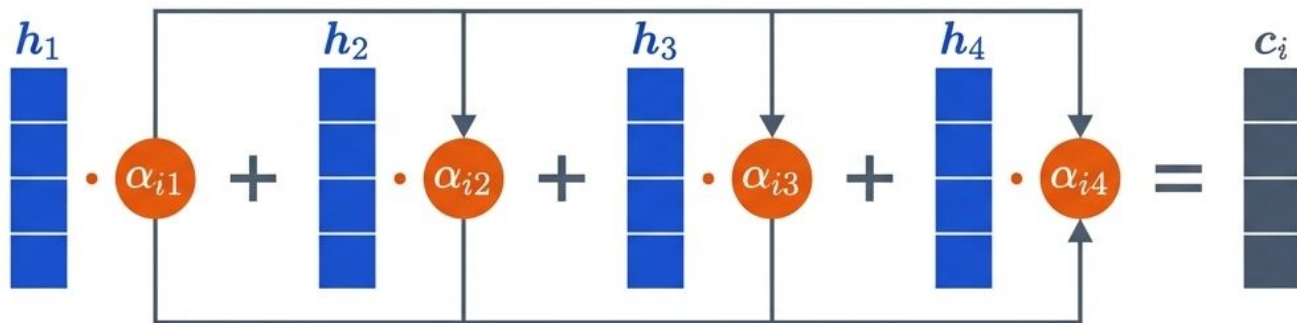
The Grand Synthesis: Architectural Divergence

Dimension	Bahdanau Attention	Luong Attention
Alignment Function	FFNN (Concat $\rightarrow$ W $\rightarrow$ tanh $\rightarrow$ $\mathbf{v}$)	Dot Product ($\mathbf{s} \cdot \mathbf{h}$)
Terminology	Additive Attention	Multiplicative Attention
State Dependency	Previous Decoder State ($\mathbf{s}_{i-1}$)	Current Decoder State ($\mathbf{s}_i$)
Architectural Injection	Input to the LSTM base	Concatenated AFTER the LSTM
Complexity & Speed	High parameters, slower training	Zero added alignment parameters, faster

Takeaway: Bahdanau solved the bottleneck by proving Attention works. Luong optimized the architecture by proving mathematical simplicity (dot products) out-competes complex neural approximation. Both pave the direct road to Self-Attention and Transformers.

The Universal Equation of Attention

$$\mathbf{c}_i = \sum_{j=1}^N \alpha_{ij} \mathbf{h}_j$$



Definition

The context vector is a weighted sum of all encoder hidden states.

The Core Challenge

How do we calculate α_{ij} ? How do we determine the exact similarity/importance between an input state and the current translation step?